Information Security

Lecture 2: The Web and Stateful Tracking

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

2 The Web and Stateful Tracking **3**
2.1 The Web in Four Objects 3
2.2 Sessions and the Cookie Attachment Rule 5
2.3 Sites, Origins, Parties 7
2.4 Cross-Site Tracking versus First-Party Analytics 9
2.5 Respawning and the Other Stores 10
2.6 Cookie Syncing and Real-Time Bidding 12
2.7 Clickjacking and Framing Defences 14
Chapter takeaways 16

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 2

The Web and Stateful Tracking

2.1 The Web in Four Objects

What are we trying to get?

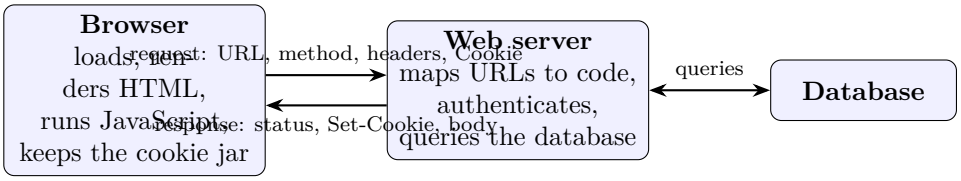
Four objects carry every argument in this chapter: the URL (what is requested), HTTP (how, and why the server forgets), HTML and JavaScript (what the browser does with the answer), and the browser's own view of the page. We also fix what "privacy" means so that "this mechanism threatens privacy" is a precise claim.

Definition 2.1.1 (Privacy)**.** Privacy concerns how information about people is *collected, used, shared and retained*, and a person's ability to *understand* what happens to their information, to *influence* how others use it, and to *avoid* unwanted observation or interference.

Reading the formula

Two halves: a data life cycle (four verbs where a tracker acts) and the person's agency (three verbs a tracker defeats). Invisible tracking defeats *understand*; tracking with no opt-out defeats *influence*; tracking that survives cookie deletion defeats *avoid*. The cases the course cites: Cambridge Analytica (Christopher Wylie, 18 March 2018: "we exploited Facebook to harvest millions of people's profiles . . . and target their inner demons"); Google's $392 million settlement with 40 US states (November 2022) over misleading location tracking.

Definition 2.1.2 (URL, HTTP, HTML)**.** In `https://informationsecurity.com:443/courses/web-security?week=5&topic=xss` the parts are the *protocol*, the *hostname* (resolved to an IP address by DNS), the *port*, the *path* and the *query string*. HTTP is an application-layer, client–server, request–response protocol: a request names a URL and carries a method (GET, POST), headers and an optional body; a response carries a status code and the resource or an error. HTTP is **stateless**: the server keeps nothing once the TCP connection closes. HTML describes structure (text, images, links, forms); JavaScript supplies logic.



The browser loads a window or frame, renders it (parsing HTML, executing scripts), and then reacts to events: user actions (`onclick`, `onmouseover`), rendering events (`onload`, `onbeforeunload`, `onerror`), timers (`setTimeout`). JavaScript reads the current URL through `location`: `href` is the complete URL including the fragment, `host` is hostname plus port, `hostname` the host alone, `pathname` the path, `search` the query. History, for completeness: Bush's Memex (1945, non-linear navigation), Berners-Lee's ENQUIRE (1980, CERN, local only), the 1989 proposal *Information Management: A Proposal* that became the WWW.

Key insight

Two facts drive everything that follows. (1) *Statelessness* means an application (a shopping cart) cannot exist unless state is added on top of HTTP; trackers reuse that added state, so tracking is the web's own design pointed at another goal. (2) *A script runs with the authority of the page that includes it*, not of the server that served it: `analytics.js` fetched from `metrics.test` but included by `shop.test` executes as `shop.test`'s code and sees `shop.test`'s cookies.

Go deeper: the details an examiner asks about

- The fragment (`#...`) is in `location.href` but is never sent in HTTP; a script must read and transmit it.
- GET fetches (a link click is a GET); POST submits forms and logins. `SameSite=Lax` (§2.2) distinguishes them.
- Stateless ≠ anonymous: every request carries the IP and headers (User-Agent, Accept-Language) that recognise a browser without state (Chapter 3).

Hard questions, answered

Q. `https://a.example/p#secret` includes `<script src="https://t.example/x.js">`. What does `t.example` learn from the request for `x.js`, and what could the script learn afterwards?
A. From the request: t.example's own cookies, the Referer `https://a.example/p` (no fragment), IP and headers. Afterwards the script, running as `a.example`, can read `location.href` including `#secret` and a.example's non-HttpOnly cookies, and send them to t.example explicitly.

2.2 Sessions and the Cookie Attachment Rule

What are we trying to get?

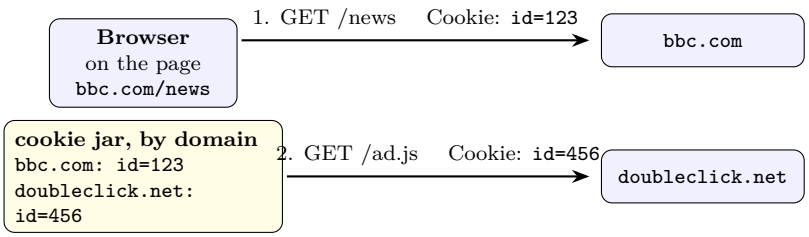
We want the two ways a stateless server can recognise a returning browser, why the first leaks, and the exact semantics of the second, cookies, because every tracking mechanism and every defence in this chapter is a consequence of one rule.

Option 1: the identifier in the URL

Generate a random token on the first visit and append it to every link: `?sessionid=8F4A92C1`. It works and it leaks: the URL is copied into bookmarks and messages, written into server logs, and transmitted in the `Referer` header of every request the page triggers, so an image loaded from a third-party server receives the full URL of the embedding page, session included.

Option 2: cookies (1994)

Definition 2.2.1 (Cookie; the attachment rule)**.** A server stores a name–value pair in the browser with a `Set-Cookie` response header. The browser persists it and *sends it back in a* `Cookie` *request header on every subsequent request to that domain, regardless of which site initiated the request.* Without `Expires`/`Max-Age` the cookie is a *session cookie*, deleted when the browser closes; with one it is *persistent*. Ending a session server-side means deleting the cookie.



Request 2 was caused by the bbc page (it embeds the script), but the jar only looks at the *destination* domain and attaches doubleclick's cookie.

Reading the formula

Both clauses of the rule are load-bearing. *To that domain*: no site can read or receive another site's cookie, so identifiers must be smuggled through URLs to cross sites (§2.6). *Regardless of initiating site*: when `bbc.com` embeds a script from `doubleclick.net`, the request to doubleclick carries doubleclick's cookie even though the user is "on" bbc; that is third-party tracking, and it is also cross-site request forgery. The jar is indexed by destination domain and does not know which page caused the request; the server cannot tell either unless it looks at the `Referer` (or modern `Sec-Fetch-*`) headers. `SameSite`, added in 2016, qualifies exactly this clause.

Attributes

Definition 2.2.2 (Scope, lifetime, security attributes)**.** `Domain`: default is the current host only; set to a parent domain (`Domain=example.com` from `websec.example.com`) to cover it and all its subdomains; cannot name an unrelated domain or a public suffix. `Path`: that path and its sub-paths. `Expires`/`Max-Age`: lifetime. `HttpOnly`: not readable through `document.cookie`; still *sent*. `Secure`: sent only over HTTPS and settable only from an HTTPS response. `SameSite=Strict`: never attached to cross-site requests. `SameSite=Lax` (browser default when absent): attached to cross-site requests only if they are *top-level navigations* with a *safe method* (GET, HEAD, OPTIONS, TRACE per RFC 7231). `SameSite=None`: attached to all cross-site requests; requires `Secure` (Chrome rejects None without Secure; Firefox and Edge warn).

Reading the formula

Each security attribute closes one door. `HttpOnly` closes *script theft*: an injected script cannot read the session cookie; it does nothing about the cookie being sent, so it does not touch CSRF or tracking. `Secure` closes *network theft*. `SameSite` closes the *cross-site attachment* door of the rule: Strict shuts it (a link from another site arrives logged out); Lax leaves it open only for an ordinary link click (top-level, GET), the case that breaks the web least; None keeps the 1994 behaviour, which a third-party tracker needs and which therefore must be HTTPS-only.

Technique: Is cookie c attached to a request from page P to destination D?

1. Is D the cookie's domain, or a subdomain of it when `Domain` was set? If not: never.
2. Does the request path start with the cookie's path?
3. If `Secure`: is the request HTTPS?
4. Do P and D have the same registrable domain (§2.3)? If yes: attached. If no (cross-site): Strict ⇒ no; Lax ⇒ only if top-level navigation *and* safe method; None ⇒ yes.

`HttpOnly` never enters: it only governs `document.cookie`.

Worked example: Quiz 1, question 4

`https://shop.example.test` sets `aid=A72; Domain=example.test; Path=/; Secure; HttpOnly; SameSite=Lax`.

A From the shop, load `https://cdn.example.test/banner.png`: `cdn.example.test` is under `example.test`; HTTPS; same site, so no cross-site question: *attached*.

B The same over `http://`: `Secure` fails: *not attached*.

C From `https://social.test`, Alice *clicks a link* to `https://shop.example.test/profile`: cross-site, but a top-level

GET navigation, which Lax permits: *attached*.

D `https://social.test` *embeds* `https://shop.example.test/tracker.png`: cross-site and not top-level: *not attached*.

Answer: A and C. `HttpOnly` was a distractor.

Gotcha

Lax has *two* conditions and both must hold: a cross-site iframe or image fails "top-level"; a cross-site POST form is top-level but fails "safe method". Only a plain link passes. Students who remember "Lax allows navigation" and forget "GET only" answer CSRF questions wrong.

Go deeper: the details an examiner asks about

- `Set-Cookie: test=1; Domain=.example.com; Secure; HttpOnly; SameSite=None`: sent on every HTTPS request to example.com and its subdomains, cross-site included, invisible to JavaScript, and still a *session* cookie (no expiry). Without `Secure`, Chrome drops it.
- Scope is by *name*: in the TP, `lab.test:8000` and `127.0.0.1:8000` are different jars for the same Flask server, which is why the hosts file is edited.
- Trackers need `None; Secure`; banks choose Strict (no CSRF at all) or Lax (inbound links arrive logged in).

Hard questions, answered

Q. A bank cookie is `HttpOnly; Secure` with no SameSite. An attacker page auto-POSTs a transfer form to the bank. Is the cookie sent?
A. Under the Lax default: no, a cross-site POST fails the safe-method condition. Treated as None (old browsers): yes, and the transfer runs as the victim. HttpOnly and Secure are irrelevant to attachment.

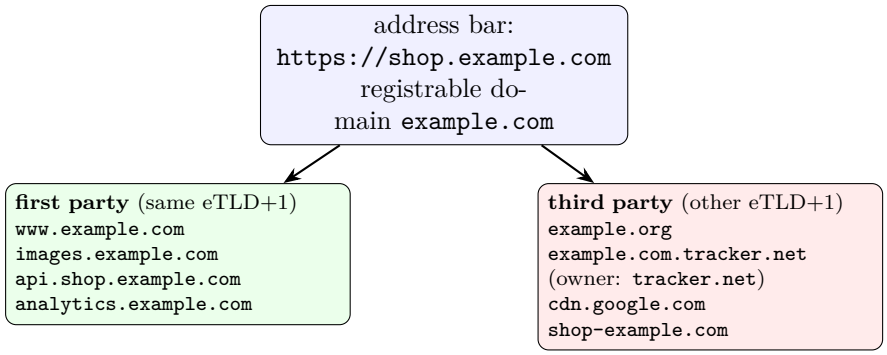
Q. Can `a.example.com` set a cookie that `b.example.com` receives? One that `example.org` receives?
A. Yes, with `Domain=example.com`; no, a different registrable domain.

Q. Does `HttpOnly` stop a tracker?
A. No: the tracker does not read its cookie with script; the browser *sends* it. HttpOnly only blocks `document.cookie`.

2.3 Sites, Origins, Parties

Definition 2.3.1 (Registrable domain; first and third party; origin)**.** The **effective top-level domain** (eTLD) is the longest public suffix a hostname ends with (`com`, `co.uk`, `github.io`); the **registrable domain** (eTLD+1) adds one label: the unit a single owner registered. A request is **first-party** when its destination has the registrable domain shown in the address bar, **third-party** otherwise; cookies are first- or third-party accordingly. An **origin** is (scheme, host, port); the **same-origin policy** lets a script read documents, storage

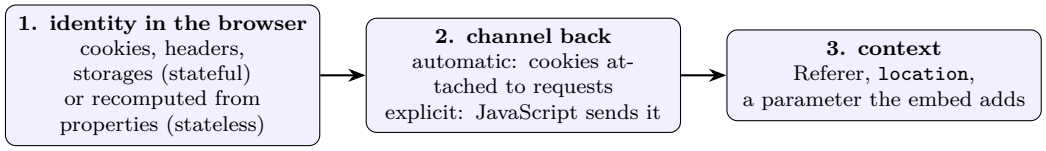
and `document.cookie` only of its own origin. Subdomains are same-site but cross-origin.



Reading the formula

Why registrable domains? Because they are the unit of *ownership*, read from the right (the `facebook.mobile.com` lesson of Chapter 1). *Site* governs cookie *attachment* (SameSite, partitioning); *origin* governs script *access*. The same-origin policy never prevents the browser from sending a cookie (Quiz 1 Q6C is false); it prevents `bbc.com`'s script from reading doubleclick's cookie.

The three capabilities of a tracker



Remove (1) and every visit is a stranger; remove (2) and the tracker never hears; remove (3) and it knows "123 is back" but not where. Chapter 3 replaces (1) by stateless recognition.

Hard questions, answered

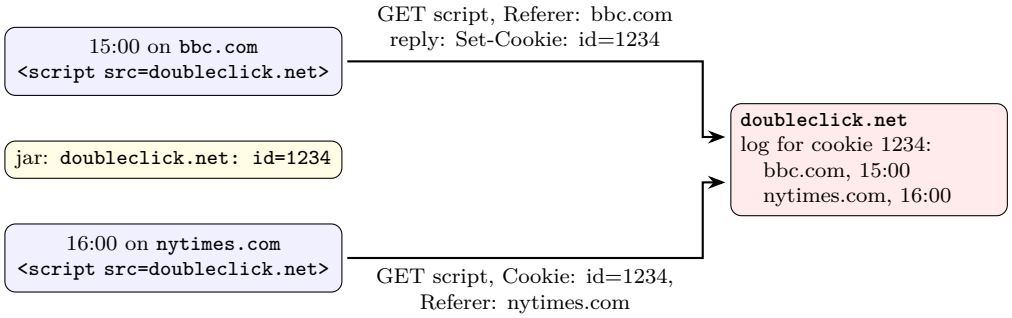
Q. Visiting `https://news.bbc.co.uk`, classify `static.bbc.co.uk`, `bbc.com`, `bbc.co.uk.cdn-edge.net`.
A. First (registrable domain `bbc.co.uk`); third (`bbc.com` is another registrable domain); third (owner `cdn-edge.net`).

Q. Can a script on `a.example.com` read `b.example.com`'s cookies? Are `Domain=example.com` cookies sent to both?
A. No (cross-origin: SOP blocks access); yes (same-site; Domain covers both). Access and attachment are different rules.

2.4 Cross-Site Tracking versus First-Party Analytics

What are we trying to get?

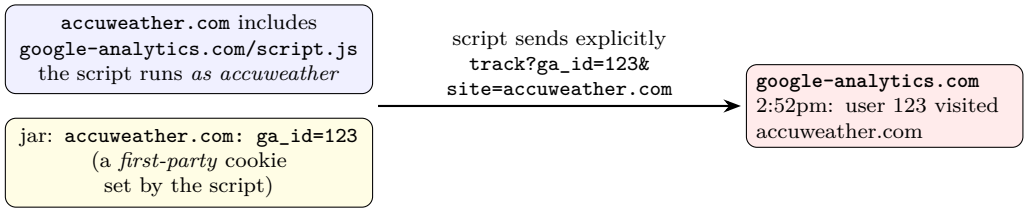
We want to derive, from the attachment rule alone, how one third-party cookie links visits across unrelated sites, and to see why the similar-looking first-party analytics cannot: the difference is under which domain the cookie is stored.



Theorem 2.4.1 (One third-party cookie)**.** *A tracker embedded on a set of sites $\mathcal{S}$ with a persistent cookie learns, per browser, the timed sequence of visits (site and page) to every site in $\mathcal{S}$, with no cooperation from the sites.*

Proof. Each visit to $s \in \mathcal{S}$ triggers a request to the tracker ("regardless of initiating site"); the request carries the tracker's cookie ("to that domain"), the same value on every site because it is stored under the tracker's domain, not under s; the Referer or an embedding parameter supplies s and the page. Grouping the log by cookie value yields each browser's history. □

First-party analytics



On nytimes.com the same script creates a *different* cookie under nytimes.com; Google receives two identifiers it cannot link.

The script file comes from Google but executes as accuweather's code (key idea of §2.1), so the cookie it creates belongs to accuweather. The browser will never attach accuweather's cookie to a request destined for google-analytics, so the script must put the value into the URL itself. Analytics therefore measures repeat visits *within* a site; it cannot collect history *across* sites.

Key insight

Tracker-domain cookie: one identifier everywhere, attached automatically. Publisher-domain cookie: one identifier per publisher, sent explicitly. "Third-party cookies blocked" blocks cookies set in a third-party context; it does not stop a third-party *script* from setting first-party cookies, which is the loophole analytics lives in.

Worked example: Quiz 1, questions 5, 6, 7

Q5 (shop stores non-HttpOnly first-party `prefs`; page loads `metrics.test/analytics.js` and an iframe from `tracker.test`): the script runs in the shop's origin and can read `prefs` (A true) and send it explicitly (D true); the iframe is another origin, SOP forbids reading (B false); the request for `analytics.js` carries metrics.test's cookies, not the shop's (C false). *Q6* (pixel from `tracker.test`, cookie `tid; SameSite=None; Secure`, third-party cookies allowed): both publishers' requests carry the same `tid` (A true, Theorem 2.4.1); the tracker links the visits (D true); publisher JavaScript cannot read it (B false); SOP does not govern sending (C false). *Q7* (third-party cookies blocked; same script on `news.test` and `shop.test`; `aid=N31` and `aid=S84`): first-party identifiers (A true); never sent automatically to metrics.test (B false); same script does not mean same identifier (C false); the script can read and send the current one (D true). Merging N31 and S84 needs an external link (a login on both, a fingerprint).

Go deeper: the details an examiner asks about

- Publishers set `HttpOnly` on their session cookie partly against the third-party scripts they include themselves.
- Citations: Roesner et al. (NSDI 2012) classified tracking behaviours; Englehardt & Narayanan (CCS 2016, OpenWPM, one million sites) measured third-party presence.

Hard questions, answered

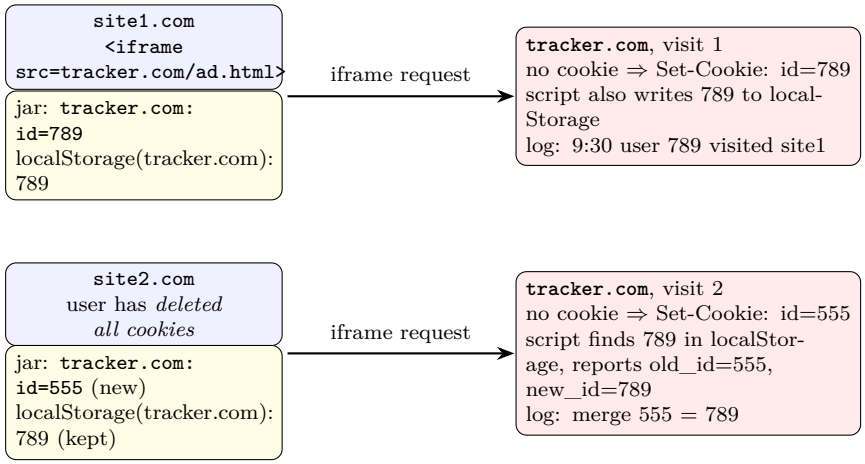
Q. Minimal change turning an analytics vendor into a cross-site tracker?
A. Have the script load a resource from the vendor's own domain that sets a vendor-domain cookie with `SameSite=None; Secure`; by Theorem 2.4.1 it links every site that includes the script. Under the Lax default the embedded request would not carry it.

2.5 Respawning and the Other Stores

What are we trying to get?

Deleting cookies is not enough: the browser has other persistent stores, and a tracker can restore the cookie from any of them. We want each mechanism, and the discipline of reading an experiment ("consistent with" is not "proves").

Definition 2.5.1 (Respawning)**. Cookie respawning** recreates a deleted cookie from a copy held elsewhere (a *zombie* cookie): HTML5 `localStorage` (string key–value pairs per origin, no expiry, across sessions), Flash local shared objects (per machine user, so across sessions *and* across browsers), HTTP cache headers `ETag` and `Last-Modified`. Evercookie (Samy Kamkar) combines many stores. Each store can also track *without* any cookie.



The iframe's script runs in `tracker.com`'s origin on every embedding site, so `tracker.com`'s localStorage is reachable from all of them: a cross-site store that cookie deletion does not touch. The user sees a new cookie value and is still the same person to the tracker.

The ETag

`ETag` is a caching mechanism: the first response to `GET A.com/i.js` carries `ETag: "x234dff"`; the browser caches the body with the tag and next time sends `If-None-Match: "x234dff"`, to which the server may answer `304 Not Modified`. Nothing forces the tag to describe the content; a tracker can make it a user identifier, and the browser will echo it on every request for that resource, even in private browsing with cookies blocked, as long as the cache lives. KissMetrics (lawsuit, August 2011) set `_km_cid=Z9iG...` as a cookie *and* as the ETag with the same value; a later request in private mode with cookies blocked still sent `If-None-Match: "Z9iG..."`. `Last-Modified` works the same way through `If-Modified-Since`.

Interpretation & discussion

Each store has its own scope and its own clearing action: cookies by domain ("clear cookies"), localStorage by origin ("clear site data"), the cache ("clear cache"); Flash objects were cleared by none of them. A privacy action is sufficient only if it clears every store the tracker might have used, which is the logic of Quiz 1 Q12.

Worked example: Quiz 1, questions 10 to 12

C = cookie, L = localStorage, E/F = ETag / fingerprint, P = server-side profile.

Test	Action before visiting	C	L	E / F	P
1	nothing	1234	1234	1234 / 88	3
2	delete the cookie	1234	1234	1234 / 88	3
3	delete cookie + localStorage	1234	new	1234 / 88	3
4	clear cookies, storage and cache	5678	5678	5678 / 88	3
5	different device	9012	9012	9012 / 42	9

Test 2: the cookie came back while localStorage still held 1234: *consistent*

with restoration from localStorage (Q10 A); it does not *prove* fingerprinting (B false), a stateful store explains it. Test 3: localStorage was wiped ("new"), yet the cookie is 1234 and the ETag still holds 1234: consistent with ETag restoration (C); the profile stayed 3, so deletion did not break the link (D false). Test 4: every store cleared, new identifiers 5678 everywhere (so the stores *were* cleared, Q11 C false), but fingerprint 88 and profile 3 persist: consistent with *stateless* relinking (B); a fingerprint identifies a browser, not a named person (A false); other browsers may share 88, so linking can produce false positives (D). Q12: the sufficient action against both respawns is to clear cookies, local storage and the relevant cache *together* (A); the other options leave a source intact.

Hard questions, answered

Q. Design a respawn that survives "clear site data" but not "clear cache".
A. Put the identifier in the `ETag` of a resource the page always loads; it returns in `If-None-Match`. Site-data clearing removes cookies and storage, not the HTTP cache.

Q. Why could Flash respawn a cookie in a *different* browser?
A. The plugin's storage was per machine user, outside any browser profile; Chrome and Firefox shared it.

Q. Everything cleared, the profile still continues. The only remaining explanation?
A. A stateless identifier recomputed from the browser's properties: Chapter 3.

2.6 Cookie Syncing and Real-Time Bidding

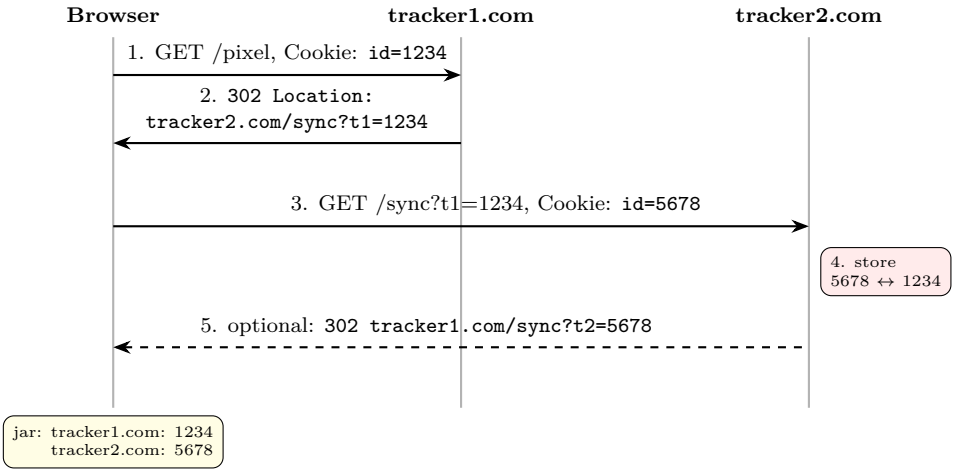
What are we trying to get?

Theorem 2.4.1 is bounded by the sites a tracker is embedded on, and two trackers hold two unrelated identifiers that neither can read. We want the protocol trick that joins them, and the advertising auction in which the trick is routine and personal data is broadcast.

The limit

Doubleclick is embedded on bbc and nytimes; adnxs on bbc and jumia. Doubleclick's log reads "bbc 15:00, nytimes 16:00"; adnxs's reads "bbc 15:00, jumia 17:00". Neither sees the union, and each has its own cookie for the browser that the other cannot receive ("to that domain"). Two partial profiles exist that nobody can join.

Definition 2.6.1 (Cookie syncing)**. Cookie syncing** is the process by which two different trackers link the identifiers they have each assigned to the same browser.

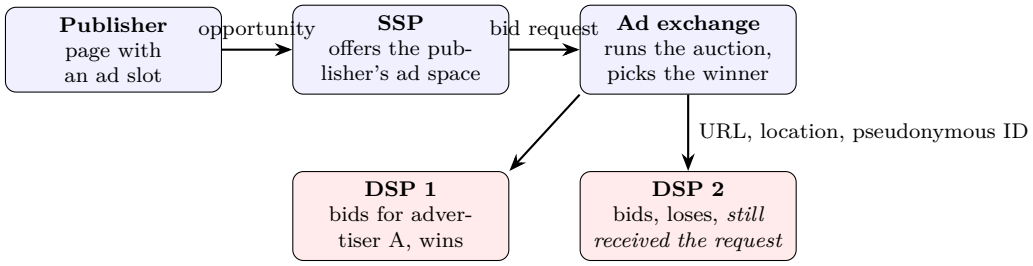


Step by step. (1) A publisher page embeds a pixel from tracker 1; the request carries tracker 1's cookie, `id=1234`. (2) Tracker 1 does not answer with an image; it answers with a redirect whose `Location` URL contains its own identifier for this browser. (3) The browser follows the redirect automatically; the new request is to tracker 2's domain, so by the attachment rule it carries *tracker 2's* cookie, `id=5678`, and the URL still carries `t1=1234`. (4) Tracker 2 now sees both values in one request and records the pair. (5) Optionally it redirects back so tracker 1 learns the mapping too. From then on both trackers can exchange data about this browser under either identifier; tracker 1's demographics (age 25, female) join tracker 2's contact data (email, address), and each tracker's coverage becomes the union of their sites.

Key insight

Nothing illegal happened at the browser level: it followed a redirect, as it must, and attached each domain's cookie to the request for that domain, as it must. The identifier crossed the domain boundary *inside the URL*, the one place the attachment rule does not police. Neither tracker ever read the other's cookie.

Real-time bidding



Real-time bidding transacts display ads in real time. The **SSP** (supply-side platform) helps publishers offer and sell ad space and forwards each opportunity to the exchange; the **ad exchange** runs the auction, distributes the bid request and selects the winner; a **DSP** (demand-side platform) evaluates opportunities and bids automatically for the users its advertisers want. The bid request (page URL, approximate location, pseudonymous identifier) is *broadcast to all DSPs*, so participating in the auction, not winning it, is what leaks. Inside RTB an SSP syncs its cookie with a DSP by exactly the redirect above, so the DSP recognises in the request a user it has profiled. Only part of the process is visible in the

browser; what each DSP retains cannot be observed from the client, which is why transparency studies are hard and regulators must audit companies.

Worked example: Quiz 1, questions 8, 9 and 16

Q8: `GET tracker-a.test/sync, Cookie: aid=A42` → `302 Location: tracker-b.test/sync?aid=A42` → `GET tracker-b.test/sync?aid=A42, Cookie: bid=B71`. Tracker B can associate A42 with B71 (A true); A42 arrives as a URL parameter, not as B's cookie (B false); A passed its identifier through the redirected URL (C true); the trackers need not share an origin, that is the point (D false). *Q9*: the SSP helps the publisher sell (A true); the losing DSP-X still received the request (B true); not only the winner (C false); the pseudonymous ID is stable and *enables* linking (D false). *Q16* asks for the drawing above: browser in the middle with both jars; request 1 carrying 1234; the 302 with 1234 in the URL; request 2 carrying 5678 plus the parameter; the mapping table at tracker 2; the optional return redirect.

Go deeper: the details an examiner asks about

- Syncing merges *coverage* and *attributes*, and moves identifiers into URLs, where they leak through Referer headers and logs.
- Andrew Lewis: "if you're not paying for the product, you are the product."

Hard questions, answered

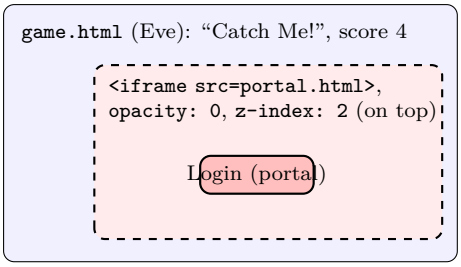
Q. Firefox gives each top-level site its own cookie jar (Chapter 3). Does that stop a sync between two trackers embedded on the *same* publisher?
A. No: on that publisher both trackers have cookies in that site's jar and the redirect links them there. Partitioning stops the *reuse* of one tracker's cookie across publishers.

Q. Why does syncing need a redirect rather than a script call?
A. Only a request *to tracker 2's domain* carries tracker 2's cookie; the redirect makes the browser issue such a request while carrying tracker 1's identifier in the URL. No script on the page can read either cookie across origins.

2.7 Clickjacking and Framing Defences

What are we trying to get?

We build the TP1 attack to see exactly which browser behaviours it relies on, then derive the defences: who must set them, and which can say "only these sites may frame me".



The portal frame sits *above* the game and is transparent; its Login button is aligned exactly over the game's button.

Alice's click goes to the top-most element: the *authenticated* portal's Login.

Alice is logged into the Student Portal. Eve's `game.html` embeds `portal.html` in an iframe positioned over the game with a higher `z-index` and `opacity: 0`; the game button moves randomly on each click, but at a chosen score it is placed exactly under the portal's Login. Alice clicks there; the click lands on the invisible Login; the portal, which received her session cookie when the iframe loaded, acts as Alice. The attack relies on three facts: a click goes to the top-most element and `opacity: 0` keeps the frame in hit-testing (`visibility: hidden` or `pointer-events: none` would not); the framed page received its cookies ("regardless of initiating site"; a Lax or Strict session cookie would *not* arrive, which mitigates); and nothing forbids framing unless the portal says so.

Definition 2.7.1 (Framing controls)**.** Both are HTTP response headers sent by the *framed* page and enforced by the browser. `X-Frame-Options: DENY` forbids all framing; `SAMEORIGIN` allows only same-origin framing; `ALLOW-FROM` is obsolete. CSP `frame-ancestors 'none'` (= DENY), `'self'` (= SAMEORIGIN), or a list of permitted origins (`frame-ancestors 'self' https://apps.um6p.ma`); it takes precedence over X-Frame-Options and is the only way to whitelist a specific third origin.

Technique: The TP's research questions

Protections: X-Frame-Options, CSP frame-ancestors (and JavaScript frame-busting, weak). Each controls who may embed the page. Same-origin only: `SAMEORIGIN` or `frame-ancestors 'self'`. A portal that must never be framed: `frame-ancestors 'none'` plus `X-Frame-Options: DENY` for old browsers. If a trusted university application must later embed it: `frame-ancestors 'self' https://trusted.um6p.ma`, which X-Frame-Options cannot express.

Hard questions, answered

Q. Setting anti-framing headers on the *game* page: useful?
A. No; the attacker controls the game page. The victim page must send them.

Q. Why is `opacity: 0` essential and `visibility: hidden` useless to the attacker?
A. Opacity keeps the frame in the hit-testing order, so it receives the click; visibility hidden removes it, so the click reaches the game.

Q. Which cookie attribute makes the clickjacked login click harmless, and why is it not the recommended defence?
A. `SameSite=Lax`/Strict on the session cookie: the cross-site iframe loads logged out. Still insufficient: actions needing no session remain hijackable and the page is still embeddable; frame-ancestors is the control.

Chapter takeaways

Summary: Stateful tracking

- Cookies are attached to every request to their domain, whoever initiated it; SameSite qualifies the second clause (Lax = top-level *and* safe method; None needs Secure); HttpOnly and Secure close script and network theft.
- Site (eTLD+1) governs attachment; origin governs script access; first party = the address bar's registrable domain.
- One tracker-domain cookie links all embedding sites (Theorem 2.4.1); publisher-domain analytics cookies cannot.
- Respawn from localStorage, Flash, ETag; clear all stores together; what survives everything is stateless.
- Syncing crosses domains inside a redirect URL; RTB broadcasts the bid request to every DSP.
- Clickjacking: transparent authenticated iframe; defence on the framed page: X-Frame-Options or CSP frame-ancestors.